

Scholarship Finder — Technical Overview

AI-powered scholarship discovery + college-admissions assessment tool. This document covers the **technology stack**, **architecture design**, and **API running costs**.

Doc set: see also CLAUDE.md (working guide), security.md (security posture), and skill.md (hard architectural rules).

1. Technology Stack

The defining choice: **zero runtime dependencies**. The entire backend is the Node.js standard library, and the frontend is plain HTML/CSS/JS with no build step or framework.

Layer	Technology	Notes
Runtime	Node.js (>=16; built/run on v24)	Pure stdlib: http, https, fs, path, crypto, net, dns, global fetch
HTTP server	http.createServer (no Express)	Routes dispatched by if (method && url) chain
Frontend	Single-page public/index.html	Inline CSS/HTML/JS; no React/Vue/bundler
Frontend helper	public/js/redesign.js (deferred)	Category pills, college map, verification badges
Static data	public/data/college-coords.json	Map pin coordinates
LLM	Anthropic Claude Sonnet 4.6	Direct HTTPS to api.anthropic.com/v1/messages; model hardcoded server-side
Web-search grounding	Tavily Search API	search_depth: "basic", max_results: 5
Persistence	In-memory Maps + local logs/*.jsonl	No database, no ORM — state resets on restart
Auth	Single ACCESS_PASSWORD + in-memory session tokens	64-char hex, HttpOnly cookie, 24h TTL
Deployment	Railway (prod) / 127.0.0.1 (local)	Single process
Dependencies	None (package.json has no dependencies)	No node_modules in production

Why zero-dependency? Smaller attack surface, no supply-chain risk, instant cold starts, trivial deploys, and no version churn. The trade-off — features like the URL-health cache and eligibility logging are implemented in-memory + JSONL files instead of a managed database.

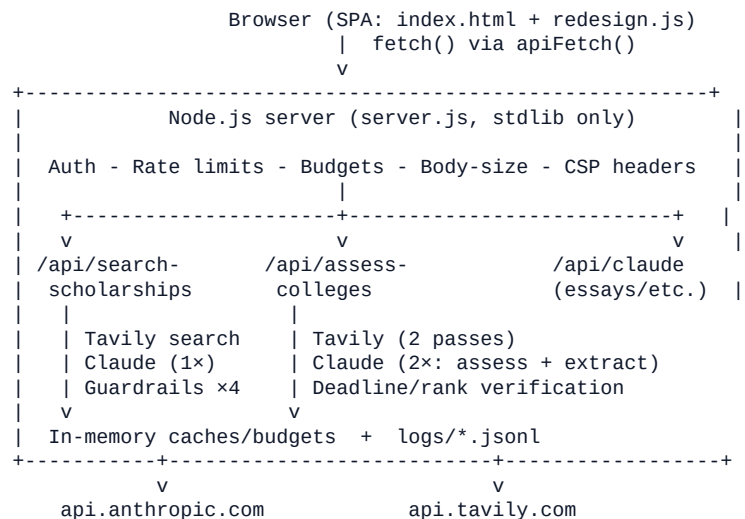
2. Architecture Design

2.1 The "AI agent" model

This is an **orchestrated LLM pipeline with tool use and a verification loop**, not a free-running autonomous agent. The agentic characteristics:

- **Tool use** — the LLM is grounded with live **Tavily web search** results rather than relying on training memory.
- **Deterministic guardrails** — server code, not the model, has final say over eligibility, deadlines, and link health (the model proposes; the server disposes).
- **A second-pass feedback loop** — for college assessment, the model's first answer is fed back through web search + a second LLM call to **verify** its deadlines/rankings before the user sees them.

2.2 High-level component diagram



2.3 Scholarship discovery pipeline (POST /api/search-scholarships)

```
profile -> 3x Tavily search (parallel, dedup, sanitize)
-> Claude prompt (grounded; structured eligibility_tags + deadline_precision)
-> SERVER-SIDE GUARDRAIL PIPELINE:
    1. Eligibility filter (drop mismatches -> logs/eligibility_drops.jsonl)
    2. Past-deadline filter (drop stale dates)
    3. Calibrated precision (exact / month / inferred; preserve amount qualifiers)
    4. URL liveness check (HEAD->GET, SSRF-guarded, conc. 8 -> url_status)
-> response + flags: grounding, droppedIneligible, droppedPastDeadline
```

The four guardrails turn "the LLM said so" into deterministically enforced output. Dead links surface a search fallback; `grounding !== 'tavily'` shows a fail-closed banner.

2.4 College assessment — two-pass verification (POST /api/assess-colleges)

Pass 1: listed schools -> Tavily (stats + deadline/ranking queries) -> Claude

(returns listed + AI-suggested matches; deadlinesVerified flags)

Pass 2 (enrichment, generic for ALL colleges):

suggested/unverified schools -> Tavily deadline search (cap 12, budget-guarded)
-> Claude extraction call (max_tokens 2,000)
-> merge verified deadlines/ranks back
-> response adds deadlineSearchedSchools -> UI shows (web) Web-verified

Best-effort and budget-guarded — if Tavily or the extraction call fails, it falls back to the first-pass values (!) unverified) and never errors.

2.5 Cross-cutting concerns

- **Security:** server-side model lock, per-IP rate limits, daily token/Tavily budgets, 32 KB body cap, CSP/security headers, path-traversal guard, prompt-injection sanitization, **SSRF protection** on all LLM-supplied URL fetches (private/metadata IPs blocked, manual per-hop redirect validation), and escHtml/escUrl output encoding. Details in security.md.
- **Statelessness:** all budgets/sessions/caches are in-memory and reset on restart. The only on-disk artifacts are gitignored JSONL logs.

3. API Running Costs

Two external paid APIs incur cost: **Anthropic (Claude)** and **Tavily**. The URL-liveness checks hit scholarship websites directly and are **free** (no API).

3.1 Pricing basis (verify current rates — see Sources)

Provider	Rate
Claude Sonnet 4.6 — input	\$3 / 1M tokens
Claude Sonnet 4.6 — output	\$15 / 1M tokens
Tavily — free tier	1,000 credits / month, no card
Tavily — pay-as-you-go	\$0.008 / credit (basic search = 1 credit)

*Prompt caching (up to ~90% off input) and batch mode (~50% off) are **not** currently used — opportunities below.*

3.2 Estimated cost per operation

Token figures are *typical* (output rarely hits the max_tokens cap), not worst-case. Tavily cost assumes the free tier is already exhausted.

Operation	Claude calls	~Tokens (in / out)	Claude \$	Tavily credits	Tavily \$	Total
Scholarship search	1	6k / 3.5k	\$0.07	3	\$0.024	~\$0.09

Operation	Claude calls	~Tokens (in / out)	Claude \$	Tavily credits	Tavily \$	Total
College assessment	2 (assess + extract)	9k / 6.5k	\$0.12	~12	\$0.10	~\$0.22
College scholarships	1	4k / 2.5k	\$0.05	~5	\$0.04	~\$0.09
Essay / strategy (/api/claude)	1	2k / 2k	\$0.04	0	\$0	~\$0.04
Combined "Find + Assess" run (findAll)	~4	~31k total	~\$0.24	~20	~\$0.16	~\$0.40

On the Tavily **free tier**, a combined run is effectively **~\$0.24** (Claude only).

3.3 Hard ceilings (server-enforced)

The app caps worst-case spend regardless of traffic:

Limit	Default	Effect
DAILY_TOKEN_BUDGET	100,000 tokens/day	~= 3 combined runs/day or ~10 scholarship-only searches/day , then 429
DAILY_SEARCH_CALLS	150 Tavily/day	scholarship-search Tavily ceiling
DAILY_COLLEGE_CALLS	60 Tavily/day	college-assessment Tavily ceiling
max_tokens caps	6k / 8k / 2k	bounds per-call output cost

Theoretical daily maximum ~= **\$1.30 Claude** (100k tokens at a blended rate)

+ **~\$1.70 Tavily** (210 credits) ~= **~\$3.00/day**.

3.4 Realistic monthly cost (friend-only MVP)

Assuming ~5 combined runs/week (~20/month):

Item	Monthly
Claude (~20 runs × ~\$0.24)	~\$5
Tavily (~400 credits)	\$0 (within 1,000-credit free tier)
Hosting (Railway hobby)	~\$0–5
Total	~= \$5–10 / month

Tavily only starts costing money past ~50 combined runs/month. The binding constraint at low volume is almost always the Claude token spend.

3.5 Cost-optimization levers (not yet applied)

1. **Prompt caching** — the large static system prompts (rankings pool, schema, guardrail rules) are re-sent every call. Caching them could cut input cost up to ~90% — the single biggest lever.
2. **Drop the 2nd college call** — replace the extraction LLM call with deterministic snippet parsing (cheaper, slightly less robust).

3. **Tighter** `max_tokens` — output dominates cost (\$15 vs \$3); trimming caps directly reduces the expensive side.
 4. **Lower** `DAILY_TOKEN_BUDGET` — the simplest hard cap on monthly spend.
-

Sources

- [Claude Sonnet 4.6 — Anthropic](#)
- [Anthropic API Pricing](#)
- [Tavily — Credits & Pricing](#)
- [Tavily — Pricing plans](#)

Cost figures are estimates based on published rates and typical token usage as of June 2026; confirm current provider pricing and your own usage before relying on them.